



Sinhgad Technical Educational Society's
SINHGAD ACADEMY OF ENGINEERING, KONDHWA

Lab Manual

Web Development (VSE-281-COM)
SE Computer
YEAR: 2025-2026

Prepared By,
Mrs.P.S.Barde
Mr.A.R.Kadam

Marking Scheme
25 Marks- Term Work
25 Marks - Oral Exam



Sinhgad Technical Educational Society's
SINHGAD ACADEMY OF ENGINEERING, KONDHWA

CERTIFICATE

This is to certify that Mr./ Miss _____, of
Class _____ Roll No. _____ has completed all the practical work in the
subject _____ satisfactorily in the Department _____
_____ as prescribed by Savitribai Phule Pune University, in the
academic year 2025 – 2026

Staff In-charge

Head of the Department

Principal

Index

Sr. No.	Assignment Title	Page No.	Date	Remark
1.	Create a simple HTML page displaying personal details using text formatting tags.			
2.	Design a web page that includes an image, hyperlink, and a nested table.			
3.	Create a form with fields: name, email, gender, date of birth, and submit button.			
4.	Apply internal, external, and inline CSS to style a web page with headings and tables.			
5.	Develop a responsive web page using Bootstrap Grid and Components.			
6.	Write a JavaScript program to validate form inputs (e.g., email, empty fields).			
7.	Create a web page that uses JavaScript to display dynamic content using DOM.			
8.	Write a JavaScript program for a simple calculator using functions and switch-case.			
9.	Design a PHP script to display "Welcome" message and current date & time.			
10.	Write a PHP program to accept form input and display it using the POST method.			
11.	Implement a PHP program for string manipulation (e.g., reverse, length, substring).			
12.	Create a PHP script to store and display values in an array.			

Assignment No.	1
Title	Create a simple HTML page displaying personal details using text formatting tags.
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 1

TITLE: Create a simple HTML page displaying personal details using text formatting tags.

OBJECTIVES:

1. To understand the basic architecture of the World Wide Web and HTML5 structure.
2. To implement semantic text formatting tags for structured document presentation.
3. To design a static web page representing a student profile (Resume/Bio-data).

PROBLEM STATEMENT:

Create a simple HTML page displaying personal details (Name, Educational Background, Address, Hobbies) using various text formatting tags to demonstrate structure and emphasis.

OUTCOME:

Students will be able to,

1. Write correct HTML5 syntax.
2. Distinguish between logical () and physical () formatting tags.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:**
 - Text Editor: Notepad++, Visual Studio Code, or Sublime Text.
 - Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

THEORY-CONCEPT:

1. Introduction to HTML:

HTML (HyperText Markup Language) is the standard markup language for documents designed to be displayed in a web browser. It is not a programming language; it is a markup language that uses "tags" to annotate text, images, and other content for display.

2. Basic Structure of an HTML5 Document:

- <!DOCTYPE html>: A declaration to the browser that this is an HTML5 document.
- <html>: The root element that wraps all content on the page.
- <head>: Contains meta-information about the document (Title, Charset, Styles) that is not visible on the page.
- <body>: Contains the visible page content.

3. Text Formatting Tags:

HTML provides tags to format text, making it readable and emphasizing important information.

- **Headings (<h1> to <h6>):**
 - <h1> defines the most important heading.
 - <h6> defines the least important heading.
 - Search engines use headings to index the structure of web pages.
- **Paragraph (<p>):** Defines a block of text. Browsers automatically add a margin before and after a paragraph.
- **Bold (vs):**
 - : Makes text bold physically (visual only).
 - : Defines text with strong importance (semantic meaning).
- **Italic (<i> vs):**
 - <i>: Displays text in italics.
 - : Indicates emphasized text (screen readers change voice tone).
- **Underline (<u>):** Underlines text.
- **Subscript (<sub>) & Superscript (<sup>):** Used for mathematical or chemical formulas (e.g., H₂OH₂O, X₂X₂).
- **Horizontal Rule (<hr>):** Creates a thematic break (a horizontal line).
- **Line Break (
):** Inserts a single line break. It is an empty tag (no closing tag).

DESIGN / EXECUTION STEPS:

1. Open your Text Editor (e.g., VS Code).
2. Create a new file and save it as assignment1.html.
3. Type the basic HTML boilerplate (<html>, <head>, <body>).
4. Inside <body>, use <h1> to center-align the main title "Student Bio-Data".
5. Use <hr> to separate sections.
6. Use <h2> for sub-headings like "Personal Info", "Education".
7. Use <p> tags for details and apply , <i>, <u> where necessary.
8. Open the file in a web browser to verify the output.

CONCLUSION / ANALYSIS:

Hence, we have learned how to install and configure tomcat server.

Assignment No.	2
Title	Design a web page that includes an image, hyperlink, and a nested table.
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 2

TITLE: Design a web page that includes an image, hyperlink, and a nested table.

OBJECTIVES:

1. To learn how to embed multimedia (Images) and manage file paths.
2. To understand the concept of Hyperlinks (Anchors) for navigation.
3. To design complex page layouts using Nested Tables.

PROBLEM STATEMENT:

Design a web page that includes a student profile photo, a hyperlink to the university website, and a nested table structure to display a weekly academic timetable.

OUTCOME:

Students will be able to,

- Organize data effectively using tables.
- Link external resources and images.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:**
 - Text Editor: Notepad++, Visual Studio Code, or Sublime Text.
 - Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

TECHNOLOGY / TOOL:

- **HTML Multimedia:** Utilizing `<img>` tags.
- **Hyperlinks:** Utilizing `<a>` tags and attributes.
- **HTML Tables:** Using `<table>`, `<tr>`, `<td>`, `<th>`.

THEORY-CONCEPT:

1. The Image Tag (`<img>`):

Images are fundamental to modern web design. The `<img>` tag is used to embed an image.

- **Attributes:**
 - o `src` (Source): The path to the image file (relative or absolute URL).
 - o `alt` (Alternative Text): Text displayed if the image fails to load. Crucial for screen readers (accessibility).
 - o `width / height`: Specifies the size of the image in pixels.
- **Note:** `<img>` is an empty tag (it does not have a closing tag).

2. Hyperlinks (<a>):

Hyperlinks allow users to navigate between pages.

- **Syntax:** Link Text
- **Attributes:**
 - href: The destination URL.
 - target: _blank opens the link in a new tab; _self opens in the same frame.

3. HTML Tables:

Tables are used to arrange data in rows and columns.

- <table>: Container.
- <tr>: Table Row.
- <th>: Table Header (Bold and centered by default).
- <td>: Table Data (Standard cell).

4. Nested Tables:

A table created inside the <td> of another table is called a "Nested Table". Before CSS layouts (Flexbox/Grid), nested tables were the primary method for creating complex page layouts (e.g., a sidebar and a content area).

DESIGN / EXECUTION STEPS:

1. Create a main table with 1 Row and 2 Columns.
2. **Column 1 (Sidebar):** Insert the Student Image () and a button wrapped in an <a> tag linking to the University site.
3. **Column 2 (Content):** Create a new <table> inside this cell for the Timetable.
4. Define headers (Time, Mon, Tue, Wed) and fill the rows with subject names.
5. Use cellpadding to create space inside cells.

CONCLUSION / ANALYSIS:

Hence, we successfully implemented a complex page layout using HTML tables. We learned how to nest tables to organize content and how to integrate multimedia and navigation links effectively.

Assignment No.	3
Title	Create a form with fields: name, email, gender, date of birth, and submit button.
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 3

TITLE: Create a form with fields: name, email, gender, date of birth, and submit button.

OBJECTIVES:

1. To understand the mechanism of client-server data transfer via Forms.
2. To implement various form controls (Text, Radio, Checkbox, Date, Select).
3. To understand the POST vs GET method concept.

PROBLEM STATEMENT:

Create a "Student Registration Form" with fields: Name, Email, Gender (Radio button), Date of Birth (Date picker), and a Submit button.

OUTCOME:

Students will be able to,

- Capture user data securely using HTML forms.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:**
 - Text Editor: Notepad++, Visual Studio Code, or Sublime Text.
 - Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

TECHNOLOGY / TOOL:

- **HTML Forms:** <form>, <input>, <label>, <select>, <textarea>.
- **HTTP Methods:** Understanding the distinction between GET and POST in the form method attribute.

THEORY-CONCEPT:

1. The <form> Element:

The <form> element acts as a container for input controls.

- **Action:** Specifies the URL (server script) where data is sent upon submission.
- **Method:**
 - o GET: Appends form data to the URL in name/value pairs. Not secure. Length limit exists.
 - o POST: Sends data in the HTTP request body. Secure. No size limit.

2. The <input> Element:

The most versatile form element. Its behavior depends on the type attribute:

- type="text": Standard text box.

- type="password": Masks characters.
- type="radio": Allows selecting **one** option from a set. (Must share the same name).
- type="checkbox": Allows selecting **multiple** options.
- type="date": Provides a calendar picker.
- type="email": Validates email format automatically.
- type="submit": Sends the form data to the server.

3. Usability:

- <label>: Defines a label for elements. Use the for attribute to bind the label to the input id.
- required: Attribute to ensure the field is not empty.
- placeholder: Hint text inside the input field.

DESIGN / EXECUTION STEPS:

1. Create the <form> container with method="POST".
2. Use a table inside the form to align labels and inputs neatly.
3. Add inputs:
 - o Text for Name.
 - o Radio buttons for Gender (Name="gender" for both).
 - o Date input for DOB.
4. Add a Submit and Reset button.

CONCLUSION / ANALYSIS:

Hence, we designed a functional HTML form. We learned how to group radio buttons using the name attribute and how to use the date input for better user experience. This form serves as the interface for future server-side processing assignments.

Assignment No.	4
Title	Apply internal, external, and inline CSS to style a web page with headings and tables.
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 4

TITLE: Apply internal, external, and inline CSS to style a web page with headings and tables.

OBJECTIVES:

1. To understand the separation of content (HTML) and presentation (CSS).
2. To demonstrate the three methods of inserting CSS: Internal, External, and Inline.
3. To apply CSS properties to styling fonts, colors, and tables.

PROBLEM STATEMENT:

Apply internal, external, and inline CSS to style a web page containing headings and tables.

OUTCOME:

Students will be able to,

- Style HTML elements efficiently.
- Understand the CSS "Cascading" logic and specificity.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:**
 - Text Editor: Notepad++, Visual Studio Code, or Sublime Text.
 - Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

TECHNOLOGY / TOOL:

- **CSS3:** Cascading Style Sheets Level 3.
- **Selectors:** Class (.), ID (#), and Element selectors.

THEORY-CONCEPT:

1. What is CSS?

CSS (Cascading Style Sheets) is a stylesheet language used to describe the presentation of a document written in HTML. It controls layout, colors, fonts, and responsiveness.

2. Three Ways to Insert CSS:

- **Inline CSS:** Placed inside the HTML tag using the style attribute.
 - o *Usage:* `<p style="color:red;">Text</p>`
 - o *Priority:* Highest.
- **Internal CSS:** Placed inside the `<style>` element within the `<head>`.
 - o *Usage:* Styling a single page.
- **External CSS:** Placed in a separate .css file and linked using `<link>`.

- o *Usage:* Best practice for styling entire websites.

3. CSS Selectors:

- **Element Selector:** Selects all instances of a tag (e.g., `p { color: red; }`).
- **ID Selector (#):** Selects a unique element (e.g., `#header { ... }`).
- **Class Selector (.):** Selects elements with a specific class (e.g., `.btn { ... }`).

4. The Box Model:

All HTML elements can be considered as boxes. The model consists of:

- **Content:** The text/image.
- **Padding:** Space between content and border.
- **Border:** A line around the padding.
- **Margin:** Space outside the border.

DESIGN / EXECUTION STEPS:

1. Create an HTML file.
2. Use **Inline CSS** on an `<h1>` tag to change its color.
3. Use **Internal CSS** (`<style>`) to style the body background and table fonts.
4. Create a CSS Class `.styled-table` to add borders and padding to the table.
5. View the result.

CONCLUSION / ANALYSIS:

Hence, we successfully applied CSS to enhance the visual appeal of the webpage. We observed that Inline CSS overrides Internal CSS, demonstrating the "Cascading" nature of stylesheets.

Assignment No.	5
Title	Develop a responsive web page using Bootstrap Grid and Components.
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 5

TITLE: Develop a responsive web page using Bootstrap Grid and Components.

OBJECTIVES:

1. To understand the principles of Responsive Web Design (RWD).
2. To utilize the Bootstrap Framework's Grid System.
3. To implement pre-built Bootstrap Components (Navbar, Cards)

PROBLEM STATEMENT:

Develop a responsive web page using Bootstrap Grid and Components that adapts its layout for Mobile and Desktop screens seamlessly.

OUTCOME:

Students will be able to,

- Build mobile-first websites.
- Use CDNs to integrate libraries.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:**
 - Text Editor: Notepad++, Visual Studio Code, or Sublime Text.
 - Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

TECHNOLOGY / TOOL:

- **Bootstrap 5:** A front-end framework.
- **CDN (Content Delivery Network):** Hosting for the library.

THEORY-CONCEPT:

1. What is Bootstrap?

Bootstrap is the most popular CSS Framework for developing responsive and mobile-first websites. It provides a collection of CSS and JavaScript code bits (components) like buttons, navbars, and modals.

2. The Grid System:

Bootstrap divides the screen width into **12 columns**.

- **.container:** Provides a fixed-width container with margins.
- **.row:** Creates a horizontal group of columns.
- **.col--:** Defines the column size.
 - o **col-md-4:** Takes 4 columns (1/3rd width) on medium screens (Desktop).

- o **col-sm-12**: Takes 12 columns (Full width) on small screens (Mobile).

3. Responsive Design:

Using these classes, elements can sit side-by-side on a desktop but automatically stack vertically on a mobile phone, providing an optimal viewing experience.

DESIGN / EXECUTION STEPS:

1. Create an HTML file.
2. Add the Bootstrap CSS link (CDN) in the <head>.
3. Add a Navbar component.
4. Create a container. Inside it, add a row.
5. Create 3 divs with class col-md-4. This ensures a 3-column layout on PC.
6. Inside these divs, add Card components.
7. Resize the browser window to see the layout adjust.

CONCLUSION / ANALYSIS:

Hence, we successfully implemented a responsive web page. On a desktop, the cards appear in a row (3 columns). On a mobile device, they stack vertically, ensuring readability.

Assignment No.	6
Title	Write a JavaScript program to validate form inputs (e.g., email, empty fields).
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 6

TITLE: Write a JavaScript program to validate form inputs (e.g., email, empty fields).

OBJECTIVES:

1. To understand the syntax, variables, and functions in JavaScript.
2. To access HTML form elements using the DOM (Document Object Model).
3. To implement logic that validates user input before it is sent to the server.

PROBLEM STATEMENT:

Write a JavaScript program to validate a Login Form. The script must ensure:

1. Fields (Email, Password) are not left empty.
2. The email address format is valid (contains '@' and '.').
3. The password meets a minimum length requirement (e.g., 6 chars).

OUTCOME:

Students will be able to,

- Write JS functions to intercept form submission.
- Use alert() to provide feedback.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:**
 - Text Editor: Notepad++, Visual Studio Code, or Sublime Text.
 - Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

TECHNOLOGY / TOOL:

- **JavaScript:** Client-side scripting language.
- **DOM API:** document.getElementById, value property.

THEORY-CONCEPT:

1. Why Validation?

Client-side validation provides a better user experience by catching errors immediately. It also reduces server load and bandwidth usage by preventing invalid requests.

2. Accessing Form Data:

To validate data, JS needs to read what the user typed.

```
var email = document.getElementById("email_id").value;
```

3. The onsubmit Event:

Forms have an onsubmit event. We attach a JS function to this.

- onsubmit="return validateForm()"
- If the function returns false, the browser **cancels** the submission.
- If the function returns true, the browser **proceeds** to the server.

4. String Methods:

- .length: Returns character count.
- .indexOf("@"): Returns the position of the character. Returns -1 if not found.

DESIGN / EXECUTION STEPS:

1. **UI Design:** Create a simple HTML Login form with Email and Password inputs. Add id attributes to them.
2. **Event Binding:** Add onsubmit="return validateForm()" to the <form> tag.
3. **Script Definition:** Inside <head>, create a <script> block. Define function validateForm().
4. **Data Retrieval:** Inside the function, create variables email and pass and assign them values from the DOM.
5. **Logic 1 (Empty Check):** if (email == "" || pass == ""). If true, alert("Empty") and return false.
6. **Logic 2 (Format):** Check if @ exists using indexOf.
7. **Logic 3 (Length):** Check pass.length < 6.
8. **Success:** If all checks pass, return true.

CONCLUSION / ANALYSIS:

Hence, we implemented client-side validation using JavaScript. This ensures that only valid data reaches the server, enhancing security and user experience.

Assignment No.	7
Title	Create a web page that uses JavaScript to display dynamic content using DOM.
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 7

TITLE: Create a web page that uses JavaScript to display dynamic content using DOM.

OBJECTIVES:

1. To understand the DOM (Document Object Model) structure as a tree of nodes.
2. To learn how to select HTML elements using JavaScript.
3. To manipulate content (innerHTML) and styles (style object) dynamically based on user interaction.

PROBLEM STATEMENT:

Create a web page that uses JavaScript to change text content, background color, and visibility of a specific DIV element based on button clicks.

OUTCOME:

Students will be able to,

- Make web pages respond to user actions without reloading.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:**
 - Text Editor: Notepad++, Visual Studio Code, or Sublime Text.
 - Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

TECHNOLOGY / TOOL:

- **DOM:** The bridge between HTML and JavaScript.
- **Properties:** innerHTML, style, display.

THEORY-CONCEPT:

1. What is the DOM?

When a web page loads, the browser creates a **Document Object Model**. It is an object-oriented representation of the web page. It allows JavaScript to access and change all the elements of an HTML document.

2. Modifying Content:

- element.innerHTML: Reads or writes the HTML content inside a tag.
 - o document.getElementById("box").innerHTML = "New Text";

3. Modifying Styles:

- `element.style.property`: CSS properties can be accessed via the style object. Note that hyphenated CSS properties become camelCase in JS.
 - CSS: background-color -> JS: backgroundColor
 - `element.style.color = "red";`

4. Modifying Visibility:

- `element.style.display`: Setting this to "none" hides the element (removes it from layout). Setting it to "block" shows it.

DESIGN / EXECUTION STEPS:

1. **HTML Structure:** Create a central div with ID targetBox. Give it initial text and some CSS borders so it's visible.
2. **Buttons:** Create three buttons: "Change Text", "Change Style", "Toggle Visibility".
3. **Function 1 (Text):** Write a JS function `changeText()`. Select the box. Set `.innerHTML` to something new.
4. **Function 2 (Style):** Write `changeStyle()`. Set `.style.backgroundColor` to yellow and `.style.borderRadius` to 50px.
5. **Function 3 (Visibility):** Write `toggleBox()`. Use an if/else statement. If display is none, make it block, else make it none.
6. **Binding:** Attach these functions to the onclick attribute of the buttons.

CONCLUSION / ANALYSIS:

Hence, we successfully manipulated the DOM tree using JavaScript. This practical demonstrates the core principle behind modern Single Page Applications (SPAs) like React or Angular, where content updates dynamically without a full-page refresh.

Assignment No.	8
Title	Write a JavaScript program for a simple calculator using functions and switch-case..
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 8

TITLE: Write a JavaScript program for a simple calculator using functions and switch-case.

OBJECTIVES:

1. To implement mathematical logic and algorithms using JavaScript functions.
2. To use Control Structures (switch-case, if-else).
3. To understand Type Coercion (Converting String inputs to Numbers).

PROBLEM STATEMENT:

Write a JavaScript program for a simple calculator that accepts two numbers and performs Addition, Subtraction, Multiplication, or Division based on the user's choice.

OUTCOME:

Students will be able to,

- Process numerical data.
- Handle logical edge cases (e.g., Division by Zero).

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:**
 - Text Editor: Notepad++, Visual Studio Code, or Sublime Text.
 - Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

TECHNOLOGY / TOOL:

- **Data Types:** HTML inputs return Strings. JS + operator on strings performs concatenation ("1"+"1"="11").
- **Parsing:** parseFloat() converts "12.5" (string) to 12.5 (number).
- **Switch Statement:** An efficient way to dispatch execution to different parts of code based on the value of an expression (the operator).

THEORY-CONCEPT:

1. What is the DOM?

When a web page loads, the browser creates a **Document Object Model**. It is an object-oriented representation of the web page. It allows JavaScript to access and change all the elements of an HTML document.

2. Modifying Content:

- element.innerHTML: Reads or writes the HTML content inside a tag.
 - o document.getElementById("box").innerHTML = "New Text";

3. Modifying Styles:

- `element.style.property`: CSS properties can be accessed via the style object. Note that hyphenated CSS properties become camelCase in JS.
 - CSS: `background-color` -> JS: `backgroundColor`
 - `element.style.color = "red";`

4. Modifying Visibility:

- `element.style.display`: Setting this to "none" hides the element (removes it from layout). Setting it to "block" shows it.

DESIGN / EXECUTION STEPS:

1. **UI:** Create a div container. Add two input type="number" fields.
2. **Buttons:** Create 4 buttons (+, -, *, /). Pass the operator as a string argument to the function: `onclick="calculate('+')"`.
3. **Function:** Create function `calculate(op)`.
4. **Parsing:** Get values from inputs and wrap them in `parseFloat()`.
5. **Validation:** Use `isNaN()` to check if inputs are valid numbers.
6. **Logic:** Use `switch(op)`. For division, add an if check to prevent division by zero.
7. **Output:** Update a `<span>` with the result.

CONCLUSION / ANALYSIS:

Hence, we built a functional tool using JavaScript logic. We learned that data types matter significantly in programming (String vs Number) and how to make code reusable using functions with parameters.

Assignment No.	9
Title	Design a PHP script to display "Welcome" message and current date & time.
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 9

TITLE: Design a PHP script to display "Welcome" message and current date & time.

OBJECTIVES:

1. To configure and understand the local server environment (XAMPP/Apache).
2. To master the syntax, file structure, and execution flow of PHP scripts.
3. To implement built-in PHP functions for dynamic Date and Time display.

PROBLEM STATEMENT:

Design a PHP script to display a "Welcome" message and the current date & time. The script must handle Timezone settings correctly to display local time.

OUTCOME:

Students will be able to,

- Distinguish between Client-Side (JS) and Server-Side (PHP) execution.
- Deploy scripts to the htdocs directory and execute them via localhost.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:** XAMPP/WAMP Server, Text Editor (VS Code), Web Browser.

TECHNOLOGY / TOOL:

- **PHP (Hypertext Preprocessor):** A server-side scripting language designed for web development.
- **Apache HTTP Server:** The web server software responsible for accepting HTTP requests and serving the PHP files.
- **XAMPP Control Panel:** A tool to manage the Apache and MySQL services.

THEORY-CONCEPT:

. Client-Side vs. Server-Side:

- **HTML/JS (Client-Side):** The code is sent to the browser, and the browser executes it. You can see the code in "View Source".
- **PHP (Server-Side):** The code stays on the server. The server processes the code (e.g., calculates math, fetches database rows) and sends **only the resulting HTML** to the browser. The user never sees the raw PHP code.

2. The Execution Environment:

Browsers cannot understand PHP tags (<?php ... ?>). Therefore, we need an intermediate software—a **Web Server** (like Apache).

- **htdocs:** This is the "root" folder of the web server in XAMPP. Any file placed here is accessible via the URL `http://localhost/filename.php`.

3. PHP Syntax Basics:

- **Tags:** All PHP code must be enclosed in <?php (start) and ?> (end).
- **Statements:** Every instruction must end with a semicolon ;.
- **Variables:** All variables start with a dollar sign \$ (e.g., \$username). PHP is loosely typed; you don't need to declare types (int/string).
- **echo:** A language construct used to output one or more strings to the client.

4. Date & Time Functions:

- `date_default_timezone_set('Region/City')`: Essential for displaying the correct local time, as servers often default to UTC.
- `date(format_string)`: formats a local timestamp.
 - d: Day (01-31)
 - M: Month (Jan-Dec)
 - Y: Year (2024)
 - h:i:s A: 12-hour format with AM/PM.

DESIGN / EXECUTION STEPS:

1. Environment Setup:

- Open the **XAMPP Control Panel**.
- Click "Start" next to **Apache**. Ensure it turns green (running on port 80/443).

2. Directory Creation:

- Navigate to your installation folder (usually `C:\xampp\htdocs\`).
- Create a new folder named `Lab_Assignment_9`.

3. File Creation:

- Open your text editor. Create a new file named `index.php`.
- *Note: PHP files must have the .php extension, not .html.*

4. Coding Logic:

- Write standard HTML5 boilerplate.
- Inside the <body>, open the PHP tag <?php.
- Set the timezone to 'Asia/Kolkata'.
- Define variables for the Welcome Message.
- Use echo to print HTML tags mixed with PHP variables.

5. Execution:

- o Open your web browser.
- o Type the URL: `http://localhost/Lab_Assignment_9/index.php`.
- o **Do not** drag and drop the file into the browser (`file://` protocol will not work).

CONCLUSION / ANALYSIS:

Hence, we successfully set up the server-side environment. We learned that PHP scripts must be accessed via the `http://localhost` protocol rather than the local file path. We also demonstrated how to mix HTML structure with PHP logic to generate dynamic content like current timestamps.

Assignment No.	10
Title	Write a PHP program to accept form input and display it using the POST method.
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 10

TITLE: Write a PHP program to accept form input and display it using the POST method.

OBJECTIVES:

1. To understand the mechanism of passing data from a Client (HTML Form) to a Server (PHP Script).
2. To differentiate between HTTP GET and HTTP POST methods regarding security and visibility.
3. To implement PHP Superglobals (\$_POST) for retrieving and sanitizing user input.

PROBLEM STATEMENT:

Write a PHP program to accept form input (Name, Age) and display it using the POST method. The system should categorize the user (e.g., Voting Eligibility) based on the input age.

OUTCOME:

Students will be able to,

- Handle form submissions securely.
- Validate and sanitize user input on the server side.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:** XAMPP/WAMP Server, Text Editor (VS Code), Web Browser.

TECHNOLOGY / TOOL:

- **PHP Superglobals:** \$_POST, \$_GET, \$_REQUEST, \$_SERVER.
- **HTTP Protocol:** Request/Response cycle headers.

THEORY-CONCEPT:

1. PHP Superglobals:

PHP uses special predefined variables called "Superglobals" that are accessible from any scope in the script.

- **\$_POST['key']:** An associative array containing data sent via the HTTP POST method. The 'key' matches the name attribute of the input field.
- **\$_GET['key']:** Contains data sent via URL parameters.

2. GET vs. POST:

Feature	GET	POST
Visibility	Data is visible in the URL bar.	Data is hidden in the HTTP Body.
Security	Low (Never use for passwords).	High (Standard for sensitive data).
Data Size	Limited (~2000 characters).	Unlimited (Server config dependent).
Use Case	Search queries, Bookmarking.	Login, Registration, Uploading.

3. Security (Sanitization):

Never trust user input. Hackers can inject HTML or JavaScript (XSS attacks) into input fields.

- `htmlspecialchars($data)`: Converts special characters to HTML entities (e.g., `<` becomes `<`), rendering the code harmless.

DESIGN / EXECUTION STEPS:

1. **File Setup:** Create `form_handler.php` in `htdocs/Lab_Assignment_10`.
2. **HTML Form Creation:**
 - o Create a `<form>` tag.
 - o Set `method="POST"`.
 - o Set `action="<?php echo $_SERVER['PHP_SELF'];?>"`. This technique allows the script to handle the submission on the **same page**.
3. **Input Definition:** Ensure every input has a unique name attribute (e.g., `name="username"`).
4. **PHP Logic Block:**
 - o Place PHP code *below* the HTML form.
 - o Check if the form was submitted: `if ($_SERVER["REQUEST_METHOD"] == "POST")`.
 - o Retrieve data: `$name = $_POST['username'];`.
 - o Apply `htmlspecialchars()` for security.
5. **Conditional Logic:** Check if the Age is `> 18`. Store a message in a variable.
6. **Display:** Use `echo` to show the result in a styled `div`.

CONCLUSION / ANALYSIS:

Hence, we successfully bridged the gap between the frontend and backend. We learned how to use the `$_POST` superglobal to retrieve user data securely. We also implemented basic sanitation practices to protect our application from malicious input.

Assignment No.	11
Title	Implement a PHP program for string manipulation (e.g., reverse, length, substring).
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 11

TITLE: Implement a PHP program for string manipulation (e.g., reverse, length, substring).

OBJECTIVES:

1. To explore PHP's extensive library of built-in String functions.
2. To perform essential data processing tasks such as searching, reversing, slicing, and formatting text.
3. To demonstrate how backend logic handles textual data.

PROBLEM STATEMENT:

Implement a PHP program to demonstrate at least 5 string manipulation functions (e.g., reverse, length, substring, uppercase, replace) on a sample string.

OUTCOME:

Students will be able to,

- Process and format textual data efficiently.
- Understand the zero-based indexing of strings in PHP.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:** XAMPP/WAMP Server, Text Editor (VS Code), Web Browser.

TECHNOLOGY / TOOL:

- **PHP String Library:** A collection of over 100 functions for string handling.

THEORY-CONCEPT:

1. Strings in Web Development:

Strings are the most common data type in web applications (Usernames, Passwords, Emails, Content). Efficient manipulation is key to performance and data integrity.

2. Key PHP String Functions:

- **strlen(\$string):** Returns the length (number of bytes) of a string.
- **str_word_count(\$string):** Returns the number of words.
- **strrev(\$string):** Reverses the string.
- **strpos(\$string, \$search):** Finds the position of the *first* occurrence of a substring. Returns false if not found.

- **str_replace(\$search, \$replace, \$subject):** Replaces all occurrences of the search string with the replacement string. (Case-sensitive).
- **strtoupper(\$string) / strtolower(\$string):** Converts text case.
- **substr(\$string, \$start, \$length):** Extracts a specific portion of the string.

DESIGN / EXECUTION STEPS:

1. **Script Setup:** Create strings.php.
2. **Input Definition:** Define a variable \$input_string with a sentence (e.g., "Web Development using PHP").
3. **Operation 1 (Length):** Call strlen(\$input_string) and store in a variable.
4. **Operation 2 (Manipulation):** Call strrev() and strtoupper().
5. **Operation 3 (Analysis):** Call strpos() to find a specific word like "PHP".
6. **Operation 4 (Modification):** Use str_replace() to change "PHP" to "Python" (just as an example).
7. **Output Formatting:** Use HTML (Unordered List) to display the results clearly.

CONCLUSION / ANALYSIS:

Hence, we explored the capabilities of PHP's string library. We learned that strings in PHP are zero-indexed and that built-in functions can perform complex tasks (like replacement and searching) in a single line of code, which is essential for data cleaning and formatting in backend development.

Assignment No.	12
Title	Create a PHP script to store and display values in an array.
Roll No.	
Class	SE Computer
Date	
Subjet	Web Development
Signature	

ASSIGNMENT 12

TITLE: Create a PHP script to store and display values in an array.

OBJECTIVES:

1. To understand the concept of Arrays in PHP as a versatile data structure.
2. To differentiate between **Indexed Arrays** (Lists) and **Associative Arrays** (Key-Value pairs).
3. To master the foreach loop for iterating through array data and rendering it in HTML.

PROBLEM STATEMENT:

Create a PHP script to store and display values in an array. The script should demonstrate:

1. An Indexed Array displayed as an unordered list.
2. An Associative Array (Subject Marks) displayed as an HTML table.

OUTCOME:

Students will be able to,

- Store complex data structures in memory.
- Generate dynamic HTML tables from array data using loops.

SOFTWARE & HARDWARE REQUIREMENTS:

1. **Hardware:** Desktop/Laptop (Intel Core i3 or higher, 4GB RAM).
2. **Software:** XAMPP/WAMP Server, Text Editor (VS Code), Web Browser.

TECHNOLOGY / TOOL:

- **PHP Arrays:** An ordered map that maps values to keys.
- **Loops:** for, foreach.

THEORY-CONCEPT:

1. What is an Array?

An array is a special variable that can hold more than one value at a time. In PHP, arrays are extremely flexible and can hold mixed data types.

2. Types of Arrays in PHP:

- **Indexed Arrays:** Arrays with a numeric index. The keys are automatically assigned (0, 1, 2...).
 - Creation: `$colors = array("Red", "Green", "Blue");` OR `$colors = ["Red", "Green"];`

- Access: \$colors[0] returns "Red".
- **Associative Arrays:** Arrays with named keys that you assign. Ideal for database records.
 - Creation: \$age = array("Peter"=>"35", "Ben"=>"37");
 - Access: \$age['Peter'] returns "35".
- **Multidimensional Arrays:** Arrays containing one or more arrays (Matrix).

3. The foreach Loop:

The most efficient way to loop through arrays in PHP.

codePHP

// For Indexed

```
foreach ($array as $value) { code... }
```

// For Associative

```
foreach ($array as $key => $value) { code... }
```

DESIGN / EXECUTION STEPS:

1. **File Setup:** Create arrays.php.
2. **Part 1 (Indexed):**
 - Declare an array \$tech containing strings ("HTML", "CSS", "PHP").
 - Add a new item dynamically using \$tech[] = "SQL";.
 - Use echo ""; to start a list.
 - Use foreach to iterate the array and print \$value.
3. **Part 2 (Associative):**
 - Declare an array \$marks with pairs ("Web Dev" => 85, "DS" => 78).
 - Use echo "<table>"; to start a table.
 - Create the table header row manually.
 - Use foreach (\$marks as \$subject => \$score) to iterate.
 - Inside the loop, print table rows <tr><td>\$subject</td><td>\$score</td></tr>.
4. **Formatting:** Add CSS to make the table look professional.

CONCLUSION / ANALYSIS:

Hence, we mastered PHP Arrays. We distinguished between Indexed arrays (for simple lists) and Associative arrays (for labeled data). We learned that foreach loops are the engine for generating dynamic HTML tables, which is the foundational logic used in all data-driven web applications (like displaying search results or e-commerce products).